

Главная мысль:

База данных — это не просто набор таблиц на диске. Это система, где данные, метаданные, буферы памяти, транзакции, блокировки, журналы, резервные копии и репликация работают вместе, чтобы данные были согласованными, надёжными и доступными.

1. Как мы создаём базу данных

1.1. Сначала не SQL, а предметная область

На практике база данных начинается не с команды `CREATE TABLE`, а с вопроса:

Какие объекты реального мира мы хотим хранить и какие связи между ними есть?

Например, делаем БД для интернет-магазина. У нас есть:

- пользователи;
- товары;
- заказы;
- позиции заказа;
- платежи;
- склады;
- остатки товаров.

То есть сначала мы выделяем **сущности**. В твоём файле сущность определяется как предмет, человек или событие, о котором хранится информация, а каждой сущности в реляционной БД обычно соответствует отдельная таблица.

Например:

```
users      -- пользователи
products   -- товары
orders     -- заказы
order_items -- строки заказа
payments   -- платежи
```

Дальше определяем связи:

```
users 1 : M orders
orders 1 : M order_items
products 1 : M order_items
orders 1 : 1 payments
```

Связи в реляционной БД реализуются через **первичные и внешние ключи**. В твоём файле прямо сказано: первичный ключ однозначно идентифицирует запись, а внешний ключ в дочерней таблице содержит значение первичного ключа родительской таблицы.

1.2. Потом создаём логическую структуру

На SQL это выглядит так:

```
CREATE DATABASE shop;
```

Потом внутри базы:

```
CREATE TABLE users (
  id BIGSERIAL PRIMARY KEY,
  email TEXT NOT NULL UNIQUE,
  name TEXT NOT NULL
);

CREATE TABLE products (
  id BIGSERIAL PRIMARY KEY,
  title TEXT NOT NULL,
  price NUMERIC(10, 2) NOT NULL CHECK (price >= 0)
);

CREATE TABLE orders (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL REFERENCES users(id),
  status TEXT NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT now()
);

CREATE TABLE order_items (
  id BIGSERIAL PRIMARY KEY,
  order_id BIGINT NOT NULL REFERENCES orders(id),
  product_id BIGINT NOT NULL REFERENCES products(id),
```

```
quantity INT NOT NULL CHECK (quantity > 0)
);
```

Здесь мы создаём не просто «таблички», а сразу задаём **правила согласованности**:

PRIMARY KEY	-- строка должна иметь уникальный идентификатор
FOREIGN KEY	-- нельзя сослаться на несуществующего пользователя/товар/ заказ
NOT NULL	-- значение обязательно
UNIQUE	-- значение не должно повторяться
CHECK	-- значение должно удовлетворять условию
DEFAULT	-- значение по умолчанию

То есть база сама начинает защищать себя от неправильных данных.

Например, если попытаться создать заказ для пользователя `user_id = 999`, которого нет, СУБД не даст это сделать, потому что нарушится внешний ключ.

1.3. Что физически происходит при CREATE DATABASE

Когда мы пишем:

```
CREATE DATABASE shop;
```

СУБД делает несколько вещей.

Во-первых, она создаёт **физическое место хранения**: каталоги, файлы данных, служебные файлы, иногда отдельные tablespace/табличные пространства. Пользователь обычно не думает о том, где именно на диске лежат данные. Это связано с принципом отчуждения данных от носителей: пользователь получает данные, не зная их физического расположения.

Во-вторых, СУБД создаёт или обновляет **системный каталог / словарь данных**. Это служебные таблицы, где хранится информация о самой базе:

```
какие есть таблицы;
какие есть столбцы;
какие типы данных у столбцов;
какие есть индексы;
какие есть ограничения;
какие есть пользователи;
```

какие есть права доступа;
какие есть представления;
какие есть процедуры;

В твоём файле это называется **ведение словаря БД**: описание данных хранится вместе с самими данными, а словарь данных — это набор таблиц с метаданными.

То есть когда мы создаём таблицу:

```
CREATE TABLE users (...);
```

СУБД не просто создаёт файл под строки таблицы. Она ещё записывает в свои системные таблицы примерно такую информацию:

```
Появилась таблица users.  
У неё есть столбцы id, email, name.  
id — первичный ключ.  
email — уникальный.  
name — NOT NULL.
```

1.4. Создание индексов

Когда мы пишем:

```
CREATE INDEX idx_orders_user_id ON orders(user_id);
```

СУБД создаёт отдельную структуру данных, обычно похожую на В-дерево. Индекс ускоряет поиск, но замедляет вставку, обновление и удаление, потому что при каждом изменении таблицы нужно обновлять ещё и индекс. В твоём файле это тоже указано: индексы ускоряют выборку, но замедляют INSERT, UPDATE, DELETE и занимают место на диске.

Важно понимать:

Индекс — это тоже объект базы данных, и его изменения тоже журналируются.

Например, если ты вставил новую строку в `orders`, СУБД должна изменить не только таблицу `orders`, но и индекс `idx_orders_user_id`.

2. Что происходит, когда мы выполняем действия в БД

Допустим, приложение отправляет запрос:

```
INSERT INTO users(email, name)
VALUES ('ivan@example.com', 'Иван');
```

На поверхности кажется: «Ну, добавилась строка».

На самом деле внутри СУБД проходит целая цепочка.

2.1. Клиент отправляет запрос серверу

В клиент-серверной архитектуре приложение не работает напрямую с файлами базы. Оно отправляет SQL-запрос серверу СУБД. В твоём файле это описано так: клиент преобразует запросы в SQL, сервер выполняет запрос и возвращает результат.

Например:

```
Web-приложение → SQL-запрос → PostgreSQL/Oracle/MySQL → результат
```

2.2. СУБД разбирает SQL

СУБД делает:

парсинг	-- понимает синтаксис запроса
проверку	-- есть ли такая таблица, такие поля, такие права
оптимизацию	-- выбирает лучший план выполнения
исполнение	-- реально читает/меняет данные

Если это `SELECT`, СУБД думает, использовать ли индекс или читать таблицу целиком.

Если это `INSERT`, `UPDATE`, `DELETE`, СУБД должна:

```
проверить ограничения;
взять нужные блокировки;
изменить данные в буфере памяти;
```

записать изменения в журнал;
позже сбросить изменённые страницы на диск.

2.3. Данные сначала часто меняются не сразу на диске, а в буферном кэше

Обычно СУБД не пишет каждую строку сразу в файл таблицы на диске. Это было бы слишком медленно.

Она держит в оперативной памяти **буферный кэш**: страницы/блоки таблиц и индексов.

Примерно так:

```
Диск:
  файл таблицы users

Оперативная память:
  страница users %123
  страница индекса users_email_idx %45
```

Когда мы вставляем пользователя, СУБД может сначала изменить страницу в памяти. Такая страница становится **грязной** — то есть она отличается от версии на диске.

Но тут возникает проблема:

| Если данные пока только в ОЗУ, а питание пропадёт, изменения исчезнут.

И вот тут появляется журнал.

3. Что происходит в журнале при действиях в БД

Журнал транзакций нужен для того, чтобы СУБД могла восстановить согласованное состояние после сбоя. В твоём файле журналирование определяется как сохранение информации, необходимой для восстановления БД в согласованное состояние при логических или физических сбоях.

Идея простая:

| Перед тем как считать изменение надёжно выполненным, СУБД должна записать след этого изменения в журнал.

Это называется принципом **write-ahead logging**:

Сначала журнал,
потом можно менять страницы данных на диске.

3.1. Что именно пишется в журнал

В твоём файле указано, что в журнале изменений фиксируются:

индикатор начала транзакции;
номер, тип и время транзакции;
пользователь;
код SQL;
имя изменяемого объекта;
предыдущее и новое состояние объекта;
индикатор фиксации завершения транзакции.

В реальных СУБД детали могут отличаться. Например, PostgreSQL в WAL обычно пишет не «полный SQL-запрос как текст», а низкоуровневые записи вида:

в такой-то странице такой-то таблицы вставлена такая-то версия строки;
в такой-то странице индекса добавлена такая-то запись;
транзакция X зафиксирована.

Но для экзамена можно говорить обобщённо:

Журнал содержит информацию, достаточную для отката незавершённых изменений и повторного применения завершённых изменений.

3.2. Пример: один INSERT

Пусть выполняется:

```
BEGIN;  
  
INSERT INTO users(email, name)  
VALUES ('ivan@example.com', 'Иван');
```

```
COMMIT;
```

Логически в журнале может появиться примерно такая последовательность:

```
BEGIN TRANSACTION tx=101

INSERT INTO users:
  новая строка:
  id = 1
  email = 'ivan@example.com'
  name = 'Иван'

UPDATE INDEX users_pkey:
  добавить ключ id = 1

UPDATE INDEX users_email_key:
  добавить ключ email = 'ivan@example.com'

COMMIT tx=101
```

Самое важное — запись `COMMIT`.

До `COMMIT` транзакция считается незавершённой. После `COMMIT` СУБД обязана обеспечить, что результат транзакции не потеряется.

3.3. COMMIT не всегда означает: «строка уже лежит в файле таблицы»

Это очень важный момент, которым строгий преподаватель может проверить понимание.

Когда транзакция завершилась:

```
COMMIT;
```

это не обязательно значит, что изменённая страница таблицы уже физически записана в файл таблицы на диске.

Чаще правильное так:

COMMIT означает, что запись о транзакции надёжно попала в журнал. А сами страницы таблиц и индексов могут быть сброшены на диск позже.

Почему так можно?

Потому что если питание пропадёт после `COMMIT`, СУБД при запуске прочитает журнал и заново «накатит» завершённую транзакцию.

То есть надёжность достигается не тем, что каждая страница таблицы мгновенно пишется на диск, а тем, что **журнал записывается раньше данных**.

4. Как работают транзакции

В твоём файле транзакция определяется как логическая единица работы из одного или нескольких SQL-операторов, которую СУБД рассматривает как неделимое действие, переводящее БД из одного согласованного состояния в другое. Там же приведён пример перевода денег: все действия выполняются целиком или не выполняются вовсе.

Классический пример:

```
Перевести 1000 рублей со счёта А на счёт В.
```

Нельзя сделать только первую половину:

```
UPDATE accounts SET balance = balance - 1000 WHERE id = 1;
```

и забыть вторую:

```
UPDATE accounts SET balance = balance + 1000 WHERE id = 2;
```

Поэтому оба действия объединяются в транзакцию:

```
BEGIN;  
  
UPDATE accounts  
SET balance = balance - 1000  
WHERE id = 1;  
  
UPDATE accounts  
SET balance = balance + 1000  
WHERE id = 2;  
  
COMMIT;
```

4.1. ACID на человеческом языке

В твоём файле свойства транзакции описаны как ACID: атомарность, согласованность, изолированность и надёжность.

Обычно на английском это:

A — Atomicity	-- атомарность
C — Consistency	-- согласованность
I — Isolation	-- изолированность
D — Durability	-- долговечность / надёжность

Atomicity — атомарность

Либо всё, либо ничего.

Если первая операция выполнялась, а вторая сломалась, СУБД должна откатить первую.

Нельзя снять деньги со счёта А и не зачислить их на счёт В.

Consistency — согласованность

Транзакция должна переводить БД из одного правильного состояния в другое правильное состояние.

Например:

Сумма денег до перевода = сумма денег после перевода.

Внутри транзакции временно может быть «неправильное» состояние, но снаружи оно не должно быть видно.

Isolation — изолированность

Параллельные транзакции не должны мешать друг другу так, будто они хаотично перемешались.

Например, два пользователя одновременно покупают последний товар. СУБД должна не допустить, чтобы оба успешно купили одну и ту же последнюю единицу.

Durability — долговечность

Если транзакция сказала `COMMIT`, то результат должен сохраниться даже после сбоя.

Именно это свойство обеспечивается журналом.

5. Транзакция, журнал и данные: один сквозной пример

Возьмём покупку товара.

Есть таблица остатков:

```
CREATE TABLE stock (  
    product_id BIGINT PRIMARY KEY,  
    quantity INT NOT NULL CHECK (quantity >= 0)  
);
```

Есть товар:

```
product_id = 10  
quantity = 5
```

Пользователь покупает 2 штуки.

```
BEGIN;  
  
UPDATE stock  
SET quantity = quantity - 2  
WHERE product_id = 10;  
  
INSERT INTO orders(user_id, status)  
VALUES (1, 'created');  
  
INSERT INTO order_items(order_id, product_id, quantity)  
VALUES (100, 10, 2);  
  
COMMIT;
```

Что происходит внутри?

5.1. Начало транзакции

СУБД создаёт идентификатор транзакции:

```
tx = 205
```

В журнале может появиться:

```
BEGIN tx=205
```

Транзакция получает свой контекст:

```
какой пользователь выполняет;  
какие объекты меняет;  
какой снимок данных видит;  
какие блокировки держит;  
какие изменения сделала.
```

5.2. Изменение остатка

Запрос:

```
UPDATE stock  
SET quantity = quantity - 2  
WHERE product_id = 10;
```

СУБД:

```
находит строку stock(product_id=10);  
проверяет, можно ли её изменить;  
берёт блокировку или создаёт новую версию строки;  
проверяет CHECK(quantity >= 0);  
меняет значение с 5 на 3;  
пишет изменение в журнал.
```

В журнале условно:

```
tx=205 UPDATE stock product_id=10:  
old quantity = 5
```

```
new quantity = 3
```

5.3. Вставка заказа

```
INSERT INTO orders(user_id, status)
VALUES (1, 'created');
```

СУБД:

```
проверяет, существует ли users(id=1);
создаёт новую строку заказа;
создаёт значение первичного ключа;
обновляет индекс первичного ключа;
пишет изменения в журнал.
```

Условно:

```
tx=205 INSERT orders:
  id = 100
  user_id = 1
  status = 'created'

tx=205 UPDATE INDEX orders_pkey:
  add key id=100
```

5.4. Вставка позиции заказа

```
INSERT INTO order_items(order_id, product_id, quantity)
VALUES (100, 10, 2);
```

СУБД проверяет внешние ключи:

```
существует ли orders(id=100);
существует ли products(id=10);
quantity > 0?
```

Если всё нормально — пишет строку и журналирует.

5.5. COMMIT

Когда доходим до:

```
COMMIT;
```

СУБД пишет в журнал:

```
COMMIT tx=205
```

И вот это ключевой момент.

После этого транзакция считается завершённой. Даже если данные ещё не полностью сброшены из буферов в файлы таблиц, журнал уже позволяет восстановить результат.

6. Что будет при ROLLBACK

Пусть внутри транзакции случилась ошибка:

```
BEGIN;  
  
UPDATE stock  
SET quantity = quantity - 2  
WHERE product_id = 10;  
  
INSERT INTO order_items(order_id, product_id, quantity)  
VALUES (999999, 10, 2);  
  
COMMIT;
```

Но заказа 999999 нет. Нарушен внешний ключ.

СУБД должна откатить транзакцию:

```
ROLLBACK;
```

Что делает журнал?

Он позволяет понять:

Было:

```
stock.quantity = 5
```

Стало внутри транзакции:

```
stock.quantity = 3
```

Но транзакция не завершилась COMMIT.

Значит, надо вернуть `stock.quantity = 5`.

Условно используется **undo** — информация для отмены незавершённых изменений.

В твоём файле сказано, что при мягком сбое журнал используется для отката незавершённых транзакций через `undo log` и наката завершённых через `redo log`.

7. Undo, redo, WAL, archived log — как не запутаться

Эти слова часто мешаются в голове. Разложим.

Undo log

Отвечает на вопрос:

Как отменить то, что сделала незавершённая или откатываемая транзакция?

Например:

```
было quantity = 5  
стало quantity = 3
```

Для отката нужно знать старое значение `5`.

Undo нужен для:

```
ROLLBACK;  
отката незавершённых транзакций после сбоя;  
иногда для MVCC/снимков старых версий.
```

Redo log

Отвечает на вопрос:

Как повторить изменения уже завершённой транзакции, если они ещё не успели попасть в файлы данных?

Например:

транзакция COMMIT была;
страница таблицы на диск не успела записаться;
после перезапуска СУБД читает redo и повторяет изменение.

Redo нужен для durability — надёжности после COMMIT.

WAL

WAL — write-ahead log, журнал предзаписи.

Принцип:

Сначала пишем запись в журнал,
потом разрешаем записывать изменённые страницы данных.

В PostgreSQL термин WAL используется напрямую. В Oracle чаще говорят redo log. Идея похожая: журнал изменений нужен, чтобы восстановить данные.

Archived log

Оперативный журнал имеет ограниченный размер. Старые сегменты журнала архивируются.

Archived log нужен для восстановления после жёсткого сбоя:

1. Берём резервную копию.
2. Накатываем archived log.
3. Получаем состояние базы ближе к моменту сбоя.

В твоём файле это прямо описано: при жёстком сбое после загрузки резервной копии выполняется накат транзакций из archived log.

8. MVCC и снимот: как транзакции не мешают читать друг другу

Вопрос про снимот у тебя отдельно вынесен в программе. В файле сказано: механизм снимотов реализует MVCC, где каждая транзакция видит снимок данных — согласованное состояние БД на определённый момент времени.

Идея такая:

Читатель не обязан ждать писателя, а писатель не обязан всегда блокировать читателя.

Допустим, есть строка:

```
product_id = 10  
quantity = 5
```

Транзакция T1 начала читать отчёт:

```
BEGIN;  
SELECT * FROM stock;
```

Она видит:

```
quantity = 5
```

Потом транзакция T2 изменила остаток:

```
BEGIN;  
UPDATE stock SET quantity = 3 WHERE product_id = 10;  
COMMIT;
```

Что должна видеть T1?

Зависит от уровня изоляции, но при snapshot-подходе она может продолжать видеть старую версию:

```
T1 видит quantity = 5  
T2 уже записала quantity = 3
```

Как так возможно?

СУБД хранит несколько версий строки:

```
старая версия: quantity = 5  
новая версия:  quantity = 3
```

У каждой версии есть служебная информация:

```
какая транзакция создала эту версию;  
какая транзакция удалила/заменила эту версию;  
видна ли эта версия текущей транзакции.
```

Поэтому изолированность достигается не только блокировками, но и версионностью.

9. Блокировки: зачем они нужны, если есть MVCC

MVCC не отменяет блокировки полностью.

Для чтения можно часто обойтись без жёстких блокировок. Но если две транзакции хотят изменить одну и ту же строку, конфликт остаётся.

Пример:

```
quantity = 5
```

T1:

```
UPDATE stock SET quantity = quantity - 3 WHERE product_id = 10;
```

T2 одновременно:

```
UPDATE stock SET quantity = quantity - 4 WHERE product_id = 10;
```

Обе не могут просто независимо изменить старое значение 5, иначе получим потерянное обновление.

Поэтому одна транзакция захватывает блокировку на изменение строки, а вторая ждёт.

Если транзакции ждут друг друга по кругу, возникает deadlock/клинч. В твоём файле клинч определяется как взаимная блокировка двух или более транзакций, когда каждая ждёт ресурс, занятый другой.

10. Что происходит при сбое

Теперь связываем журнал и восстановление.

10.1. Мягкий сбой

Мягкий сбой — например, выключили питание. Диск цел, но ОЗУ потеряно.

До сбоя было:

T1: COMMIT был, но страница данных ещё не записана на диск.
T2: изменения были, но COMMIT не было.

После перезапуска СУБД смотрит журнал.

Она делает:

1. Находит завершённые транзакции.
2. Накатывает их заново через redo.
3. Находит незавершённые транзакции.
4. Откатывает их через undo.

В твоём файле восстановление после мягкого сбоя описано именно так: загрузка ближайшей точки согласованности, откат незавершённых транзакций по undo log, накат завершённых по redo log и повторный запуск транзакций, выполнявшихся в момент сбоя.

Главная идея:

COMMIT был → изменение должно сохраниться.
COMMIT не было → изменение должно исчезнуть.

10.2. Жёсткий сбой

Жёсткий сбой — повреждены файлы данных на диске.

Тут одного оперативного журнала мало. Нужна резервная копия.

Схема:

1. Берём последнюю полную резервную копию.
2. Восстанавливаем файлы данных.
3. Берём архивные журналы.
4. Накатываем все завершённые транзакции после момента резервной копии.
5. Получаем состояние БД на нужный момент.

Это называется восстановление с использованием backup + archived logs.

11. Как происходит копирование базы данных

В твоём файле указаны три способа копирования:

```
копирование средствами ОС;  
копирование устройств/СХД;  
копирование средствами СУБД, например через согласованный снапшот.
```

Там же подчёркнуто: если копировать файлы средствами ОС, СУБД нужно остановить, иначе можно получить несогласованную копию.

Почему?

11.1. Почему нельзя просто скопировать папку работающей БД

Представь, база работает. В это время:

```
одна транзакция меняет таблицу orders;  
другая меняет индекс orders_pkey;  
третья уже сделала COMMIT, но часть страниц ещё в ОЗУ;  
четвёртая ещё не сделала COMMIT, но какие-то грязные страницы уже могли  
попасть на диск.
```

Ты в этот момент делаешь:

```
cp -r /var/lib/postgresql/data /backup
```

Что может получиться?

файл таблицы скопировался на момент 12:00:01;
файл индекса – на момент 12:00:03;
часть журнала – на момент 12:00:02;
часть изменений была только в памяти и не попала в копию;
часть незавершённых изменений могла попасть в копию.

В итоге резервная копия может оказаться внутренне противоречивой:

таблица говорит, что строка есть;
индекс её не знает;

или индекс ссылается на строку,
которой в таблице нет;

или есть изменения транзакции,
у которой нет COMMIT.

Поэтому простое файловое копирование безопасно только при остановленной СУБД:

остановили СУБД;
все буферы сброшены;
нет активных транзакций;
файлы согласованы;
копируем;
запускаем СУБД.

Это называется холодная резервная копия.

11.2. Горячая резервная копия

В реальности останавливать большую БД неудобно. Поэтому используют горячее копирование средствами СУБД.

Идея:

БД продолжает работать,
но СУБД фиксирует момент согласованности
и гарантирует, что копия + журналы позволят восстановиться.

Упрощённая схема:

1. СУБД создаёт контрольную точку / backup-состояние.
2. Фиксирует позицию в журнале: например LSN.
3. Начинается копирование файлов данных.
4. Пока файлы копируются, транзакции продолжают работать.
5. Все изменения за время копирования пишутся в журнал.
6. В backup сохраняется информация: с какой позиции журнала надо начинать восстановление.
7. При восстановлении берётся копия файлов и накатываются журналы.

То есть горячий backup — это не просто «копия файлов». Это:

копия файлов данных
+
набор журналов изменений
+
информация о позиции в журнале

Без журналов такая копия может быть неполной.

12. Важное различие: снимок транзакции и снимок резервного копирования

Слово «снимок» может означать разные вещи.

Снимок транзакции

Это логический снимок видимости данных для конкретной транзакции.

T1 видит базу такой, какой она была в момент начала T1.

Это связано с MVCC и изоляцией.

Снимок для backup

Это согласованная точка, от которой можно сделать резервную копию.

Сделали снимок файлов/устройства/БД,
а потом при необходимости накатываем журналы.

То есть:

снэпшот транзакции → нужен для изоляции;
снэпшот backup → нужен для копирования и восстановления.

Строгий преподаватель может спросить: «Снэпшот — это копия всей базы?»

Ответ:

Не всегда. В контексте транзакций снэпшот — это прежде всего согласованное представление данных для транзакции. В контексте резервного копирования снэпшот — это согласованная точка/копия состояния, пригодная для восстановления.

13. Как настраивается репликация

В твоём файле репликация определяется как поддержание в синхронном состоянии копий одних и тех же данных на нескольких узлах сети. Там же указано, что механизмы репликации строятся на журналах изменений: на основном сервере формируются журналы, они передаются на репликационный сервер и применяются на репликах.

Общая идея:

primary/master → журналы изменений → replica/standby

13.1. Репликация начинается с начальной копии

Нельзя просто сказать реплике:

«С этого момента применяй новые изменения»

Потому что у неё ещё нет старых данных.

Поэтому сначала делается **базовая копия**:

1. На основном сервере есть БД.
2. С неё снимается согласованная копия.
3. Эта копия переносится на сервер-реплику.
4. Вместе с копией фиксируется позиция журнала.

Позиция журнала нужна, чтобы реплика понимала:

Вот до этого момента данные уже есть в базовой копии.
А начиная с этой позиции надо применять новые журналы.

13.2. Потом реплика начинает читать журналы

После базовой копии основной сервер продолжает работать:

```
INSERT ...  
UPDATE ...  
DELETE ...  
COMMIT ...
```

Все изменения попадают в журнал.

Реплика получает эти журналы и применяет у себя:

```
primary:  
  tx=300 UPDATE users ...  
  tx=300 COMMIT  
  
replica:  
  получить запись журнала  
  воспроизвести UPDATE  
  увидеть COMMIT  
  сделать изменение видимым
```

То есть реплика не «придумывает» изменения сама. Она повторяет историю изменений мастера.

13.3. Почему на реплике обычно нельзя писать

В твоём файле прямо сказано: изменения на мастере повторяются на репликах, но не наоборот; `INSERT`, `UPDATE`, `DELETE` выполняются только на мастере, а `SELECT` — на мастере и репликах.

Почему так?

Пусть на мастере:


```
product_id = 10  
quantity = 5
```

На реплике кто-то локально изменил:

```
quantity = 100
```

А потом с мастера пришло:

```
quantity = 3
```

Что правильно?

```
3?  
100?  
сложить?  
перезаписать?  
создать конфликт?
```

Чтобы не решать такие конфликты, классическая схема master-replica делает реплики read-only.

14. Что происходит с транзакциями при репликации

Очень важный момент:

Репликация должна передавать не просто отдельные изменения строк, а границы транзакций.

Почему?

Допустим, на мастере была транзакция:

```
BEGIN;  
  
UPDATE accounts SET balance = balance - 1000 WHERE id = 1;  
UPDATE accounts SET balance = balance + 1000 WHERE id = 2;  
  
COMMIT;
```

Если реплика применит только первое изменение, а второе ещё нет, она временно увидит неправильное состояние.

Поэтому нормальная репликация учитывает транзакционные границы:

```
BEGIN tx=500
изменение 1
изменение 2
COMMIT tx=500
```

На реплике изменения должны стать видимыми как единая транзакция.

То есть:

До применения COMMIT:
транзакция не должна считаться завершённой.

После применения COMMIT:
все изменения транзакции становятся результатом.

15. Синхронная и асинхронная репликация

Асинхронная репликация

Схема:

1. Клиент отправил COMMIT на master.
2. Master записал COMMIT в свой журнал.
3. Master ответил клиенту: успешно.
4. Потом master отправил журналы на replica.

Плюс:

быстрее;
меньше задержка для пользователя.

Минус:

если master умер сразу после COMMIT,
а журнал ещё не успел дойти до replica,

часть последних транзакций может потеряться при переключении.

Синхронная репликация

Схема:

1. Клиент отправил COMMIT.
2. Master записал COMMIT в журнал.
3. Master отправил журнал на replica.
4. Replica подтвердила получение/запись/применение.
5. Только потом master ответил клиенту: успешно.

Плюс:

меньше риск потери подтверждённых транзакций.

Минус:

COMMIT медленнее;
если replica тормозит, master тоже может ждать.

Преподаватель может спросить:

«Когда транзакция считается зафиксированной при синхронной репликации?»

Ответ:

Это зависит от настройки: можно ждать только получения журнала репликой, записи журнала на диск реплики или полного применения изменений на реплике. Чем строже режим, тем надёжнее, но медленнее.

16. Физическая и логическая репликация

В твоём файле перечисляются типы репликации: поблочная/WAL, построчная, логическая, репликация моментальных снимков.

Разложим проще.

Физическая репликация

Передаются низкоуровневые изменения страниц/блоков.

Примерно:

```
в файле такой-то таблицы,  
в странице №123,  
изменились такие-то байты.
```

Плюсы:

```
точная копия primary;  
хорошо для standby-сервера;  
обычно быстрее и проще для аварийного восстановления.
```

Минусы:

```
реплика обычно должна быть той же СУБД и той же версии/совместимой версии;  
сложнее выбирать только часть таблиц.
```

Логическая репликация

Передаются логические изменения:

```
INSERT INTO orders ...  
UPDATE users SET ...  
DELETE FROM ...
```

Или не сам SQL, а событие уровня строки:

```
в таблице users строка id=10 изменила email.
```

Плюсы:

```
можно реплицировать отдельные таблицы;  
можно переносить данные между разными версиями;  
можно использовать для интеграций.
```

Минусы:

сложнее;
есть больше ограничений;
не всё DDL автоматически передаётся;
конфликты сложнее.

17. Как репликация связана с восстановлением

Репликация и восстановление похожи тем, что оба используют журнал.

Восстановление

backup + archived logs → восстановленная БД

Репликация

base backup + поток WAL/redo → живая реплика

То есть реплика — это как база, которая постоянно находится в режиме восстановления и всё время накатывает новые журналы с основного сервера.

Хорошая формулировка для экзамена:

Репликация фактически использует тот же принцип, что и восстановление после резервной копии: есть начальное согласованное состояние и непрерывный поток журналов изменений, которые применяются к этому состоянию.

18. Полная цепочка на одном примере

Представим интернет-магазин.

Этап 1. Создание БД

```
CREATE DATABASE shop;  
CREATE TABLE users (...);  
CREATE TABLE products (...);  
CREATE TABLE orders (...);  
CREATE TABLE order_items (...);
```

```
CREATE INDEX ...;  
GRANT ...;
```

Что делает СУБД:

- создаёт файлы/пространства хранения;
- создаёт записи в словаре данных;
- создаёт таблицы;
- создаёт индексы;
- создаёт ограничения;
- фиксирует часть этих операций в журнале;
- настраивает права.

Этап 2. Обычная работа

Пользователь делает заказ.

```
BEGIN;  
UPDATE stock SET quantity = quantity - 2 WHERE product_id = 10;  
INSERT INTO orders ...;  
INSERT INTO order_items ...;  
COMMIT;
```

Что делает СУБД:

- проверяет права;
- проверяет ограничения;
- берёт блокировки или создаёт версии строк;
- меняет данные в буферах;
- пишет undo/redo/WAL-записи;
- на COMMIT фиксирует транзакцию в журнале;
- позже сбрасывает грязные страницы на диск.

Этап 3. Если произошёл сбой

После перезапуска:

- транзакции с COMMIT → накатить;
- транзакции без COMMIT → откатить;

получить согласованное состояние.

Этап 4. Если делаем backup

Холодный backup:

```
остановить СУБД;  
скопировать файлы;  
запустить СУБД.
```

Горячий backup:

```
зафиксировать backup-точку;  
копировать файлы во время работы;  
сохранять журналы изменений;  
при восстановлении взять копию + накатить журналы.
```

Этап 5. Если настраиваем репликацию

```
сделать начальную копию primary;  
запомнить позицию журнала;  
передать копию на replica;  
запустить получение WAL/redo;  
реплика применяет журналы;  
SELECT можно отправлять на реплику;  
INSERT/UPDATE/DELETE – обычно только на master.
```

19. Что ответить строгому преподавателю

Вопрос: «Почему нельзя просто скопировать файлы работающей БД?»

Потому что в момент копирования часть данных может быть:

```
уже на диске;  
только в буфере памяти;
```

в журнале, но не в файле таблицы;
в файле таблицы, но не согласована с индексом;
частью незавершённой транзакции.

Поэтому копия может быть несогласованной. Нужно либо остановить СУБД, либо делать согласованный backup средствами СУБД.

Вопрос: «Что важнее при COMMIT — запись данных в таблицу или запись в журнал?»

Запись в журнал.

Если COMMIT попал в надёжный журнал, данные можно восстановить даже если страницы таблицы ещё не сброшены на диск.

Вопрос: «Зачем нужен undo, если есть redo?»

Redo повторяет завершённые транзакции.

Undo отменяет незавершённые или откатываемые транзакции.

redo → «повтори то, что должно быть»
undo → «убери то, чего не должно быть»

Вопрос: «Чем backup отличается от репликации?»

Backup — это копия на определённый момент времени, обычно для восстановления.

Репликация — это постоянно обновляемая копия, куда непрерывно передаются изменения.

backup: восстановиться после потери данных;
replica: повысить доступность, распределить чтение, быстро переключиться
при аварии.

Вопрос: «Можно ли считать реплику резервной копией?»

Не совсем.

Реплика защищает от отказа сервера, но плохо защищает от логической ошибки.

Например:

```
DELETE FROM users;  
COMMIT;
```

Если это ушло на реплику, данные удалятся и там.

Поэтому нужны и репликация, и backup.

Хороший ответ:

Репликация повышает доступность, но не заменяет резервное копирование. Backup нужен для возврата к прошлому состоянию, а реплика обычно повторяет текущее состояние мастера, включая ошибочные изменения.

Вопрос: «Что происходит с незавершённой транзакцией на реплике?»

Она не должна стать видимой как завершённая.

Реплика применяет изменения с учётом границ транзакций. Пока нет записи `COMMIT`, транзакция не считается зафиксированной.

Вопрос: «Почему реплика может отставать?»

Потому что между master и replica есть задержки:

```
master записал журнал;  
журнал надо передать по сети;  
replica должна записать его у себя;  
replica должна применить изменения.
```

Если изменений много или сеть медленная, возникает replication lag.

Вопрос: «Что такое точка согласованности?»

Это момент, когда состояние данных можно считать внутренне согласованным: все завершённые транзакции учтены, незавершённые не видны как завершённые, а журнал позволяет довести систему до корректного состояния.

При восстановлении СУБД обычно берёт последнюю точку согласованности, затем делает redo завершённых транзакций и undo незавершённых.

20. Самая короткая связка всей темы

Можно запомнить так:

1. Проектируем БД:
сущности → таблицы → ключи → связи → ограничения → индексы.
2. Создаём БД:
CREATE DATABASE / CREATE TABLE / CREATE INDEX.
СУБД создаёт физические файлы и записи в словаре данных.
3. Работаем с данными:
каждый INSERT/UPDATE/DELETE выполняется внутри транзакции.
4. Транзакция:
BEGIN → изменения → COMMIT или ROLLBACK.
5. Журнал:
фиксирует, что менялось, кем, когда, в какой транзакции,
что было до и что стало после.
6. COMMIT:
результат транзакции становится обязательным для сохранения.
7. Сбой:
завершённые транзакции накатываются через redo,
незавершённые откатываются через undo.
8. Backup:
берём согласованную копию данных,
а для актуализации используем журналы.
9. Репликация:
сначала делаем начальную копию,

потом непрерывно передаём и применяем журналы изменений.

10. Главная цель:

БД должна оставаться согласованной, надёжной и доступной.

И самая экзаменационная формулировка:

Транзакции задают логические границы изменений, журнал фиксирует эти изменения и их завершение, восстановление использует журнал для возврата к согласованному состоянию, а копирование и репликация используют согласованную копию и поток журналов, чтобы создать резервную или актуальную копию базы.